Part 2: Python Programming Basics

In this lesson, we learn basic Python programming concepts used in data processing.

Topics:

- Variables and data types
- Mathematical operations
- Type conversion
- String operations
- Lists
- Dictionaries

In [1]:
```
name = "Himat"
age = 35
height = 1.75
is_student = False
print(name)
print(age)
print(height)
print(is_student)
```

```
Himat
35
1.75
False
```

In [2]:
```
print(type(name))
print(type(age))
print(type(height))
print(type(is_student))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

In [3]:
```
hospital = "Hospital_A"
patient_id = 101
test_value = 5.6
test_completed = True

print(type(hospital))
print(type(patient_id))
print(type(test_value))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
```

In [4]:
```
print(type(test_completed))
```

```
<class 'bool'>
```

2. Mathematical Operations

Python supports basic mathematical operations.

These operations are useful for calculations and data analysis.

In [5]:
```
a = 20
b = 6

print(a + b) # Addition
print(a - b) # Subtraction
print(a * b) # Multiplication
print(a / b) # Division
print(a // b) # Floor division
print(a % b) # Remainder
print(a ** b) # Power
```

```
26
14
120
3.3333333333333335
3
2
64000000
```

In [6]:
```python
# Example 2
price = 100
quantity = 5

total = price * quantity

print("Total Price:", total)
```

```
Total Price: 500
```

In [7]:
```python
# Example 3
value1 = 10
value2 = 20
value3 = 30

average = (value1 + value2 + value3) / 3

print("Average:", average)
```

```
Average: 20.0
```

3. Type Conversion

Type conversion means changing data from one type to another.

Common functions:

- int() converts to integer
- float() converts to decimal number
- str() converts to string

Type conversion is important when cleaning datasets.

In [11]:
```
age = "35"

print(type(age))
age = int(age)

print(age)
print(type(age))
```

```
<class 'str'>
35
<class 'int'>
```

In [12]:
```
price = "125.50"

price = float(price)

print(price)
print(type(price))
```

```
125.5
<class 'float'>
```

In [13]:
```
patient_id = 1001

patient_id = str(patient_id)

print(patient_id)
print(type(patient_id))
```

```
1001
<class 'str'>
```

In [14]:
```
value = "12.5"

# int(value) will produce an error

value = float(value)

print(value)
```

```
12.5
```

4. Working with Strings

Strings are used to store text data.

Python provides functions to clean and transform text.

Common operations:

- upper()
- lower()
- title()
- strip()
- replace()
- split()

In [ ]:

In [16]:
```
hospital = "Hospital A"

print(hospital.upper())
print(hospital.lower())
print(hospital.title())
```

```
HOSPITAL A
hospital a
Hospital A
```

In [18]:
```
hospital = " Hospital A "

print(hospital.strip())
hospital = "Hospital_A"

print(hospital.replace("_", " "))
```

```
Hospital A
Hospital A
```

In [21]:
```
record = "Hospital_A,101,Glucose,5.6"
print(type(record))
values = record.split(",")

print(values)
print(type(values))
```

```
<class 'str'>
['Hospital_A', '101', 'Glucose', '5.6']
<class 'list'>
```

In [22]:
```
text = "Data Processing with Python"

print(len(text))
```

```
27
```

5. Lists

A list stores multiple values in one variable.

Lists are useful for storing collections of data.

We can:

- Access elements
- Add new elements
- Remove elements
- Update values
- Sort values

In [26]:
```
# Section 5 List
# Example 1
hospitals = ["Hospital_A", "Hospital_B", "Hospital_C"]
print(hospitals)
```

```
print(type(hospitals))
# hostpital positions
print(hospitals[0])
print(hospitals[1])
print(hospitals[-1])
# Add to
```

```
['Hospital_A', 'Hospital_B', 'Hospital_C']
<class 'list'>
Hospital_A
Hospital_B
Hospital_C
```

In [28]:
```
# Add to  list append command
hospitals.append("Hospital_D")
print(hospitals)
# Remove from the list
hospitals.remove("Hospital_B")
print(hospitals)
```

```
['Hospital_A', 'Hospital_B', 'Hospital_C', 'Hospital_D', 'Hospital_D']
['Hospital_A', 'Hospital_C', 'Hospital_D', 'Hospital_D']
```

In [29]:
```
# Remove on index
hospitals[0] = "Hospital_Main"

print(hospitals)
```

```
['Hospital_Main', 'Hospital_C', 'Hospital_D', 'Hospital_D']
```

In [30]:
```
# Example 2 List len, min, max,
test_values = [5.6, 7.2, 4.8, 6.1, 8.3]
print(test_values)
print(len(test_values))
```

```
[5.6, 7.2, 4.8, 6.1, 8.3]
5
```

In [31]:
```
print(min(test_values))
print(max(test_values))
print(sum(test_values))
```

```
# Exa
```

```
4.8
8.3
32.0
```

In [32]:
```
# Example 3 Average
average = sum(test_values) / len(test_values)
print("Average test value:", average)
```

```
Average test value: 6.4
```

In [33]:
```
# Example 4 sort
test_values.sort()
print(test_values)
```

```
[4.8, 5.6, 6.1, 7.2, 8.3]
```

6. Dictionaries

A dictionary stores data using key-value pairs.

Dictionaries are useful for representing structured records.

For example, one hospital laboratory record can contain:

- Hospital name
- Patient ID
- Test name
- Test value

In [34]:
```
patient = {
    "hospital": "Hospital_A",
    "patient_id": 101,
    "test_name": "Glucose",
    "test_value": 5.6
}
```

```
print(patient)
```

```
{'hospital': 'Hospital_A', 'patient_id': 101, 'test_name': 'Glucose', 'test_value': 5.6}
```

In [35]:
```
print(type(patient))
```

```
<class 'dict'>
```

In [36]:
```
print(patient["hospital"])
print(patient["patient_id"])
print(patient["test_value"])
```

```
Hospital_A
101
5.6
```

In [38]:
```
patient["test_value"] = 6.2

print(patient)
patient["unit"] = "mmol/L"

print(patient)
```

```
{'hospital': 'Hospital_A', 'patient_id': 101, 'test_name': 'Glucose', 'test_value': 6.2, 'unit': 'mmol/L'}
{'hospital': 'Hospital_A', 'patient_id': 101, 'test_name': 'Glucose', 'test_value': 6.2, 'unit': 'mmol/L'}
```

In [40]:
```
#print(patient.keys())
#print(patient.values())
```

```
dict_values(['Hospital_A', 101, 'Glucose', 6.2, 'mmol/L'])
```

In [41]:
```
print(patient.items())
```

```
dict_items([('hospital', 'Hospital_A'), ('patient_id', 101), ('test_name', 'Glucose'), ('test_value', 6.2), ('unit', 'mmol/
L')])
```

7. Combining Lists and Dictionaries

Real-world datasets contain multiple records.

We can store multiple dictionaries inside a list.

Each dictionary represents one record.

In [42]:
```
patients = [
    {"patient_id": 101, "hospital": "Hospital_A", "test_value": 5.6},
    {"patient_id": 102, "hospital": "Hospital_B", "test_value": 7.2},
    {"patient_id": 103, "hospital": "Hospital_C", "test_value": 4.8}
]

print(patients)
print(patients[0])
```

```
[{'patient_id': 101, 'hospital': 'Hospital_A', 'test_value': 5.6}, {'patient_id': 102, 'hospital': 'Hospital_B', 'test_value': 7.2}, {'patient_id': 103, 'hospital': 'Hospital_C', 'test_value': 4.8}]
{'patient_id': 101, 'hospital': 'Hospital_A', 'test_value': 5.6}
```

In [43]:
```
print(patients[1])
print(patients[0]["hospital"])
print(patients[1]["test_value"])
```

```
{'patient_id': 102, 'hospital': 'Hospital_B', 'test_value': 7.2}
Hospital_A
7.2
```

8. Introduction to Pandas DataFrames

Pandas is a Python library used for data processing and analysis.

A DataFrame stores data in rows and columns.

We can convert Python dictionaries and lists into a DataFrame.

In [51]:
```
import pandas as pd
df = pd.DataFrame(patients)

#df
df.tail(1)
```

Out[51]:

	patient_id	hospital	test_value
2	103	Hospital_C	4.8

In [54]:
```
#df.shape
#df.columns
df.dtypes
```

Out[54]:
```
patient_id      int64
hospital          str
test_value    float64
dtype: object
```

In [62]:
```
#df["hospital"]
#df[["patient_id", "test_value"]]
#df["test_value"].mean()
#df["test_value"].min()
#df["test_value"].max()
hospital = "Hospital_A"
patient_id = 105
test_value = "7.8"
df["hospital"]
```

Out[62]:
```
0    Hospital_A
1    Hospital_B
2    Hospital_C
Name: hospital, dtype: str
```

9. Practice Exercise

Complete the following tasks:

1. Create variables for hospital name, patient ID, and test value.
2. Check the data type of each variable.
3. Convert a test value from string to float.
4. Create a list containing five test values.
5. Calculate the average test value.

6. Create a dictionary containing patient information.
7. Convert the dictionary into a Pandas DataFrame.

Task 1: Check data types

Task 2: Convert test_value to float

Task 3: Create a list of test values

Task 4: Calculate average

Task 5: Create a dictionary

Task 6: Convert dictionary into DataFrame